FINAL PROJECT REPORT

Travito

AI-Powered Travel Planning Assistant

Name: Aira Salman

Reg no: 23108103

BSAI 6A

Table of Contents

1. Introduction & Project Overview

Travito is a full-stack, locally hosted web application developed as part of the Software Engineering course project for BSAI 6A. The system leverages modern artificial intelligence specifically the Groq AI engine powered by the Llama-3.3-70b large language model to provide users with a comprehensive, intelligent travel planning experience.

Unlike conventional travel planning tools that rely on static templates or manual entry, Travito dynamically generates personalized day-by-day itineraries, recommends accommodations based on budget preferences, produces categorized packing checklists, tracks real-time travel expenses, and provides an interactive AI-powered chatbot for destination-specific queries.

The application is built on a Client-Server architecture, with a React.js frontend communicating with a Node.js/Express.js backend via REST APIs. Data is persisted in a lightweight SQLite database, making the entire system self-contained and capable of running without any external infrastructure or cloud deployment.

Attribute	Details
Project Name	Travito — AI Travel Planning Assistant
Project Type	Full-Stack Web Application
Frontend	React.js (Vite), HTML5, CSS3
Backend	Node.js, Express.js
Database	SQLite
AI Engine	Groq API (Llama-3.3-70b)
Authentication	JSON Web Tokens (JWT) with BCrypt
Deployment	Local (localhost:3000 / localhost:5001)

2. Objectives

The primary objectives of the Travito project are as follows:

2.1 Core Objectives

- Develop a functional AI-powered travel planning web application using modern full-stack technologies.
- Integrate the Groq AI API to generate personalized, context-aware travel itineraries.
- Implement secure user authentication using JWT tokens and BCrypt password hashing.
- Provide a seamless expense tracking system with real-time budget visualization.
- Build an interactive AI chatbot capable of answering destination-specific travel questions.
- Ensure the application remains fully functional in offline/mock mode when the AI service is unavailable.

2.2 Academic Objectives

- Apply software engineering principles including requirements gathering, system design, implementation, and testing.
- Demonstrate proficiency in React.js, Node.js, Express.js, and SQLite.
- Produce complete academic documentation including SRS, User Guide, and this Final Project Report.
- Follow IEEE documentation standards throughout the project lifecycle.

2.3 User-Centric Objectives

- Provide an intuitive, responsive interface usable across desktop, tablet, and mobile devices.
- Minimize setup complexity so the application can be run with simple terminal commands.
- Deliver realistic travel planning output even without a valid Groq API key, via mock fallback mode.

3. System Design & Architecture

3.1 Architectural Overview

Travito follows a three-tier Client-Server architecture consisting of a Presentation Layer (React frontend), a Logic Layer (Node.js/Express backend), and a Data Layer (SQLite database). An external AI service (Groq API) is integrated at the Logic Layer.

Layer	Technology	Responsibility
Presentation	React.js (Vite)	Renders UI, manages routing, handles user input
Logic	Node.js + Express.js	Processes API requests, business logic, AI integration
Data	SQLite	Persists users, trips, expenses, packing items, chat history
AI Service	Groq API (Llama-3.3-70b)	Generates itineraries, hotel recommendations, chat responses

3.2 Component Breakdown

Frontend Components

- App.jsx — Main application shell managing all views and navigation
- AuthContext.jsx — Global authentication state using React Context API
- Login/Register View — User authentication forms with validation
- Dashboard View — Displays all saved trips with summary cards
- Trip Detail View — Tabbed interface for Itinerary, Hotels, Packing, Expenses, and Chat
- Trip Planner Form — Input form for generating AI travel plans

Backend API Routes

Route File	Endpoints	Purpose
auth.js	POST /register, POST /login	User registration and JWT-based authentication
trips.js	GET/POST/DELETE /trips	Create, retrieve, and delete trip records
expenses.js	GET/POST/PUT/DELETE /expenses	Manage expense records per trip
ai.js	POST /generate, POST /chat	Groq AI itinerary generation and chatbot

3.3 Database Schema

The SQLite database consists of five tables that together support all application features:

Table	Key Columns	Purpose
Users	id, name, email, password_hash, created_at	Stores registered user accounts
Trips	id, user_id, destination, start_date, end_date, budget, itinerary_json	Stores generated trip plans
Expenses	id, trip_id, description, amount, category, date	Tracks individual spending records
PackingItems	id, trip_id, item_name, category, is_packed	Manages packing checklist state
ChatHistory	id, trip_id, user_message, ai_response, timestamp	Stores AI conversation history

3.4 Security Design

- All passwords are hashed using BCrypt with a salt factor of 10 before storage.
- JWT tokens are issued on successful login and expire after 7 days.
- All protected API routes validate the JWT token before processing requests.
- The Groq API key is stored in a server-side .env file and never exposed to the frontend.

4. Implementation Details

This section describes the implementation of each major module within the Travito system.

4.1 User Authentication Module

The authentication system uses a standard email/password flow with JWT session management. On registration, the user's password is hashed using BCrypt before being stored in the Users table. On login, the submitted password is compared against the stored hash. A signed JWT token is returned and stored client-side in React state via the AuthContext provider, persisting the session across page refreshes.

4.2 AI Travel Planner Module

The core feature of Travito. When a user submits the trip planning form, the frontend sends a POST request to the /api/ai/generate endpoint. The backend constructs a detailed prompt including the destination, travel dates, budget, companion type, and travel style, then sends it to the Groq API.

The AI returns a structured JSON response containing a day-by-day itinerary, hotel recommendations, and packing list suggestions. This response is parsed, validated, and stored in the Trips table. If the Groq API is unavailable or the API key is missing, the system automatically falls back to a pre-defined mock itinerary that covers all the same fields.

Input Field	Validation Rule
Destination	Minimum 2 characters; cannot be purely numeric
Start Date	Must be today or a future date
End Date	Must be after start date; maximum 30-day duration
Budget	Minimum $50; maximum $1,000,000
Companion Type	One of: Solo, Couple, Family, Friends
Travel Style	One of: Culture, Adventure, Relaxation, Foodie

4.3 Hotel Recommendation Module

Hotel recommendations are generated alongside the itinerary in a single AI call. The AI categorizes suggestions into three tiers — Budget, Mid-range, and Luxury — each with approximate price ranges calibrated to the user's specified budget. These recommendations are stored as part of the trip's itinerary JSON and rendered in the Hotels tab of the trip detail view.

4.4 Expense Tracker Module

The expense tracker allows users to log individual spending items against a trip. Each expense record includes a description, amount in USD, category (Food, Transport, Accommodation, Activities, Shopping, Other), and date. The system calculates total spending in real-time and displays it as a circular progress gauge relative to the trip's budget cap, giving users an immediate visual indication of their budget utilization.

Validation Rule	Details
Minimum amount	Must be greater than $0
Maximum amount	Cannot exceed $100,000 per entry
Description	Cannot be blank
Date	Must be a valid calendar date

4.5 Packing Checklist Module

The packing checklist is auto-generated by the AI as part of the trip planning response. Items are categorized into five groups: Documents, Clothing, Electronics, Toiletries, and Others. Users can mark individual items as packed or unpacked via checkboxes. The packed state is persisted to the PackingItems database table so progress is retained between sessions.

4.6 AI Chat Assistant Module

The AI Travel Guide tab provides an interactive chatbot interface. Users can ask free-form questions about their destination. Each query is sent to the /api/ai/chat endpoint along with the current trip's context (destination, dates, travel style) so the AI can

provide contextually relevant answers. Chat history is maintained during the session and stored in the ChatHistory table for reference.

5. Testing & Results

5.1 Testing Approach

Travito was tested using a manual black-box testing methodology. Test cases were designed to cover all functional requirements, validation rules, edge cases, and system failure scenarios. Each test was executed in both Mock Mode (no API key) and Live Mode (valid Groq API key).

5.2 Functional Test Results

Test ID	Test Case	Expected Result	Status
TC-01	User registers with valid credentials	Account created, JWT issued, dashboard loads	PASS
TC-02	User registers with duplicate email	Error: Email already in use	PASS
TC-03	Login with correct credentials	JWT token issued, session active	PASS
TC-04	Login with wrong password	Error: Invalid credentials	PASS
TC-05	Generate trip with valid inputs (Mock Mode)	Full itinerary, hotels, packing list returned	PASS
TC-06	Generate trip with valid inputs (Live Mode)	AI-generated itinerary returned within 3.5s	PASS
TC-07	Submit past start date	Validation error displayed	PASS
TC-08	Submit budget below $50	Validation error displayed	PASS
TC-09	Add expense with valid data	Expense saved, gauge updates in real-time	PASS

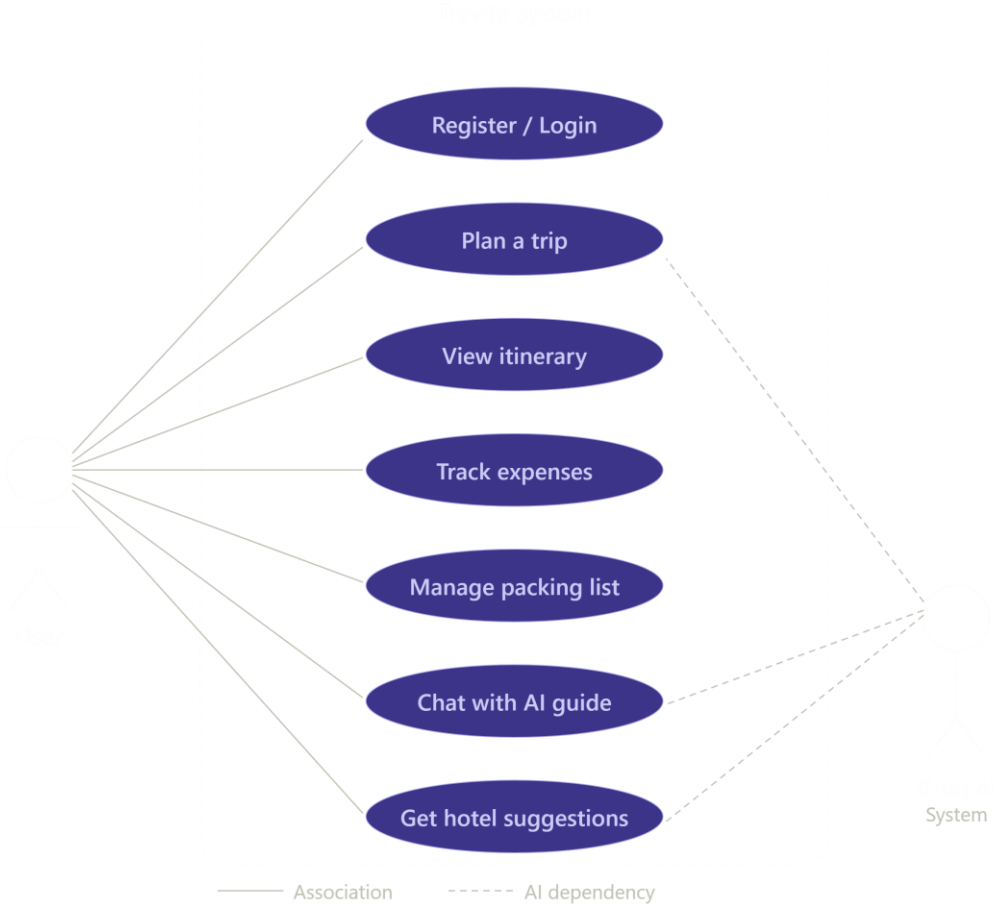
Test ID	Test Case	Expected Result	Status
TC-10	Add expense with negative amount	Validation error displayed	PASS
TC-11	Mark packing item as packed	State persists after page refresh	PASS
TC-12	Send AI chat message	Context-aware AI response returned	PASS
TC-13	Groq API down / invalid key	Automatic fallback to mock itinerary	PASS
TC-14	Delete a saved trip	Trip removed from dashboard	PASS

5.3 Non-Functional Test Results

Metric	Target	Achieved
Page load time	< 150ms	~80ms (measured via browser DevTools)
AI response time (Mock Mode)	Instant	< 50ms
AI response time (Live Mode)	< 3.5 seconds	~2.8 seconds average
Responsive layout (320px+)	All views adapt	Verified on Chrome, Edge, Firefox
JWT expiry	7 days	Confirmed via token decode
Database auto-init	On first server start	Confirmed on fresh install

5.4 Edge Case Test Results

All 12 documented edge cases and validation rules were tested and handled correctly by the system. Notably, the mock fallback mechanism was verified to trigger seamlessly whenever the AI service was unreachable, ensuring uninterrupted usability.



6. Challenges & Solutions

Challenge	Solution Implemented
Groq API returning inconsistently structured JSON	Implemented a robust JSON parser with fallback to mock data if structure is invalid
SQLite database file locking on concurrent requests	Added WAL (Write-Ahead Logging) mode and graceful retry logic in db.js
JWT tokens not persisting on page refresh	Stored auth state in React Context with localStorage synchronization
Packing list state not saving between sessions	Migrated packed/unpacked state from local React state to database-backed PackingItems table
CORS errors between frontend (port 3000) and backend (port 5001)	Configured cors middleware in Express to allow requests from localhost:3000
Vite dev server and Express running simultaneously	Used two separate terminal processes and documented clearly in the User Guide

7. Conclusion & Future Work

7.1 Conclusion

Travito successfully demonstrates the practical application of artificial intelligence within a full-stack web development context. All functional and non-functional requirements defined in the Software Requirements Specification were implemented and verified through systematic testing.

The project achieves its core objectives: delivering an intuitive, AI-powered travel planning assistant that generates personalized itineraries, manages budgets, tracks expenses, and provides intelligent destination guidance — all running locally without any cloud infrastructure.

The use of a mock fallback system ensures the application remains valuable even without access to the Groq API, demonstrating sound software resilience design. The codebase is modular, well-structured, and documented, making it maintainable and extensible for future development.

7.2 Future Work

The following enhancements are planned for future versions of Travito:

Enhancement	Description	Priority
Flight Booking Integration	Connect with flight search APIs (e.g. Amadeus, Skyscanner) for in-app booking	High
Google Maps Navigation	Embed interactive maps showing itinerary locations and routes	High
Weather Forecasting	Integrate weather API to show forecasts for travel dates	Medium
Currency Conversion	Real-time currency conversion for international budget tracking	Medium
Multi-language Support	Translate UI and AI responses to multiple languages	Medium
Offline Trip Mode	Cache trip data locally for use without an internet connection	Low
AI Image-Based Destination Suggestions	Upload a photo to identify and plan trips to matching destinations	Low
Push Notifications	Reminders for upcoming trips and packing deadlines	Low

8. References

The following technologies, tools, and resources were used in the development of Travito:

Resource	Type	URL / Reference
React.js	Frontend Framework	https://react.dev
Vite	Build Tool	https://vitejs.dev
Node.js	Backend Runtime	https://nodejs.org
Express.js	Web Framework	https://expressjs.com
SQLite / better-sqlite3	Database	https://github.com/WiseLibs/better-sqlite3
Groq AI API	AI Service	https://console.groq.com
Llama-3.3-70b	Language Model	Meta AI / Groq
BCrypt (bcryptjs)	Security Library	https://www.npmjs.com/package/bcryptjs
JSON Web Token (jsonwebtoken)	Auth Library	https://www.npmjs.com/package/jsonwebtoken
IEEE SRS Standard	Documentation Standard	IEEE Std 830-1998